

```

1
2
3 SET GLOBAL general_log = 'OFF';
4 SET GLOBAL general_log_file = '/var/log/mysql/hello-general-log.log';
5 SET GLOBAL general_log = 'ON';
6
7 [mysqld] I
8 general_log_file = /var/log/mysql/hello-general-log.log
9 general_log = 1

```

```

2
3 SET GLOBAL slow_query_log = 'OFF';
4 SET GLOBAL long_query_time = 0; # time in seconds
5 SET GLOBAL slow_query_log_file = '/var/log/mysql/slow_query_log_file_name.log';
6 SET GLOBAL slow_query_log = 'ON';
7
8 [mysqld] I
9 long_query_time = 0; # time in seconds
10 slow_query_log_file = /var/log/mysql/slow_query_log_file_name.log
11 slow_query_log = 1

```

```

[mysqld]
log_error=/var/log/mysql/mysql_error.log

```

Binary Logging

- ▶ Why should you use binary log?
- ▶ Replication:
 - ▶ You stream the changes made to a server to another server using binary logs. The slave acts as a mirror copy and can be used to distribute the load. The server that accepts the writes is referred to as a master and the mirror copy server is referred to as a slave.
- ▶ Point-in-time recovery:
 - ▶ Suppose you take a backup at 00:00 on Sunday and your database crashed at 8:00 on Sunday. Using backup, you can recover till 00:00 Sunday. Using binary logs you can reply to them, to recover till 08:00.

Using binary logging

- ▶ To enable binlog, you have to set `log_bin` and `server_id` and restart the server. You can mention the path and base name in the `log_bin` itself.
- ▶ For example, `log_bin` is set to `/data/mysql/binlogs/bin_log_name`, the binary logs are stored in the `/data/mysql/binlogs` folder with the name `bin_log_name.000001`, `bin_log_name.000002`, and so on.
- ▶ The server creates a new file in the series each time it starts or flushes the logs or the current log's size reaches `max_binlog_size`. It maintains the `server1`.
- ▶ Index file, which contains the location of each binary log.

Enable/Disable binary logs

▶ Enabling binary logs

- ▶ `log_bin = /data/mysql/binlogs/server1`
- ▶ `server_id = 100`
- ▶ `SHOW VARIABLES LIKE 'log_bin%';`
- ▶ Display all the binary logs of the server `SHOW MASTER LOGS;` or `SHOW BINARY LOGS;`
- ▶ To check current binary log position `SHOW MASTER STATUS;`

▶ Disabling binary logs for a session

- ▶ To disable `SET SQL_LOG_BIN = 0;`
- ▶ To enable set back `SET SQL_LOG_BIN = 1;`

Move to the next log

- ▶ You can use the FLUSH LOGS command to close the current binary log and open a new one:
- ▶ SHOW BINARY LOGS;
- ▶ FLUSH LOGS;
- ▶ SHOW BINARY LOGS

Expire binary logs

- ▶ Binary logs consume a lot of space based on the number of writes. Leaving them as-is can fill up the disk in no time. It is essential to clean them up.
- ▶ Set the expiry of logs using binlog_expire_logs_seconds and expire_logs_days.
 - ▶ SET @@global.expire_logs_days=2
 - ✚ Setting the value to 0 disables automatic expiry
- ▶ To purge the logs manually, execute PURGE BINARY LOGS TO '<file_name>'.
 - ▶ PURGE BINARY LOGS TO 'server1.000004';

Binary log format

- ▶ Binary logs can be written in three formats:
 - ▶ **STATEMENT:** Actual SQL statements are logged.
 - ▶ **ROW:** Changes made to each row are logged. For example, an update statement updates 10 rows, the updated information of all 10 rows is written to the log. Whereas in statement-based replication, only the update statement is written. The default format is ROW.
 - ▶ **MIXED:** MySQL switches from STATEMENT to ROW as and when needed.
- ▶ `SET @@GLOBAL.BINLOG_FORMAT = 'STATEMENT';`
- ▶ `SET @@GLOBAL.BINLOG_FORMAT = 'ROW';`

Extracting statements from a binary log

shd -v -vv

- ▶ You can use the **mysqlbinlog** utility (shipped along with MySQL) to extract the contents from binary logs and apply them to other servers.
- ▶ **server id:** Is the `server_id` value of the server where the event originated (200 in this case).
- ▶ **end_log_pos:** Is the start position of the next event.
- ▶ **thread_id:** Indicates which thread executed the event.
- ▶ **exec_time:** Is the time spent executing the event, on a master server. On a slave, it is the difference of the end execution time on the slave minus the beginning execution time on the master. The difference serves as an indicator of how much replication lags behind the master.
- ▶ **error_code:** Indicates the result from executing the event. Zero means that no error occurred.

Extracting based on time and position

- ▶ You can extract the partial data from the binary log by specifying the position. Suppose you want to do point-in-time recovery. Assume a `DROP Table` command was executed at 2020-12-22 12:18:00 and the latest available backup was 2020-12-22 12:00:00, which you already restored. Now, you need to restore data from 12:00:01 till 2020-12-22 12:17:00. Remember, if you extract the full log, it will also contain the DROP Table command and it will wipe your data again.
- ▶ `mysqlbinlog /var/lib/mysql/binlog.000017 --start-datetime="2020-12-20 00:00:01" --stop-datetime="2020-12-20 12:17:00" > binlog_extract`

Extracting from Binary Log

- ▶ **Extracting based on the database:**
- ▶ Using the `--database` option, you can filter events of a specific database. If you give this multiple times, only the last option will be considered. This works very well for row-based replication. But for statement-based replication and MIXED, this gives output only when the default database is selected.
- ▶ `sudo mysqlbinlog /data/mysql/binlogs/server1.000001 --database=employees > binlog_extract`
- ▶ **Rewriting a database name:**
- ▶ Suppose you want to restore the binarylog of the employees database on a production server as employees_dev on a development server. You can use the `--rewritedb='from_name->to_name'` option. This will rewrite all occurrences of from_name to to_name.
- ▶ `--rewrite-db='employees->employees_dev'`

Extracting from Binary Log

- ▶ **Disabling a binary log for recovery:**

- ▶ While restoring binary logs, if you don't want the mysqlbinlog process to create binary logs, you can use the **--disable-log-bin** option so that binary logs won't be written:

- ▶ shell> sudo mysqlbinlog /data/mysql/binlogs/binlog.000001 --start-position=123 --stop-position=1223 **--disable-log-bin** > binlog_restore I

- ▶ **Displaying events in a binary log file:**

- ▶ You can also use the SHOW BINLOG EVENTS command to display the events.
- ▶ SHOW BINLOG EVENTS IN 'binlog.000017';

Taking backups using mysqldump

- ▶ The mysqldump utility comes along with the mysql binary, so you do need to install it separately.
- ▶ Syntax
 - ▶ shell> mysqldump [options]
- ▶ In the options, you can specify the username, password, and hostname to connect to the database
 - ▶ -u <user_name> -p<password>

Taking backups using mysqldump

Full backup of all databases:

- ▶ Shell> `mysqldump --all-databases > dump.sql`
 - ▶ The `--all-databases` option takes a backup of all databases and all tables. The `>` operator redirects the output to the dump.sql file.
- ▶ To include stored routines and events in a dump made using `--all-databases`, use the `--routines` and `--events` options
 - ▶ To include the routines and events:
 - ▶ shell> `mysqldump --all-databases --routines --events > dump.sql`
- ▶ To get point-in-time recovery, you should specify `--single-transaction` and `--master-data`.
 - ▶ shell> `mysqldump --all-databases --routines --events --single-transaction --master-data > dump.sql`

Taking backups using mysqldump

- ▶ Specific databases and tables
 - ▶ `mysqldump --databases employees > employees_backup.sql`
- ▶ Multiple databases
 - ▶ shell> `mysqldump --databases db1 db2 db3 > dump.sql`
- ▶ To back up only a specific table
 - ▶ `mysqldump --databases employees --tables titles > titles_backup.sql`
- ▶ Ignore tables
 - ▶ `mysqldump --databases employees --ignore-table=employees.salaries > employees_backup.sql`

If you want to cause the dump file to force a drop of each database before recreating it, use the `--add-drop-database` option as well. In this case, `mysqldump` writes a `DROP DATABASE` statement preceding each `CREATE DATABASE` statement.

Taking backups using mysqldump

► Specific rows

- `shell> mysqldump --databases employees --tables employees --databases employees --tables employees --where="hire_date>'2020-01-01'" > employees_after_2020.sql`

► Backup from a remote server

- Sometimes, you may not have SSH access to the database server (as in the case of cloud instances such as Amazon RDS). In those cases, you can use mysqldump to take a backup from a remote server to the local server.

- `mysqldump --all-databases --routines --events --triggers --hostname remote_hostname > dump.sql`

► Only schema and no data

- `shell> mysqldump --all-databases --routines --events --triggers --no-data > schema.sql`

► Only data and no schema

- `shell> mysqldump --databases employees --no-create-db --no-create-info --complete-insert > data.sql`

mysqldump Tips

► Making a Copy of a Database

- `shell> mysqldump db1 > dump.sql`
- `shell> mysqladmin create db2`
- `shell> mysql db2 < dump.sql`

- Do not use `--databases` on the `mysqldump` command line because that causes `USE db1` to be included in the dump file, which overrides the effect of naming `db2` on the `mysql` command line.

mysqldump Tips

► Copy a Database from one Server to Another

- `shell> mysqldump db1 > dump.sql`
- `shell> mysqladmin create db1`
- `shell> mysql db1 < dump.sql`

► Dumping Stored Programs

- `--events`: Dump Event Scheduler events
- `--routines`: Dump stored procedures and functions
- `--triggers`: Dump triggers for tables

The `--triggers` option is enabled by default so that when tables are dumped, they are accompanied by any triggers they have. The other options are disabled by default and must be specified explicitly to dump the corresponding objects. To disable any of these options explicitly, use its skip form: `--skip-events`, `--skip-routines`, or `--skip-triggers`.

mysqldump Tips

► Dumping Table Definitions and Content Separately

- The `--no-data` option tells **mysqldump** not to dump table data, resulting in the dump file containing only statements to create the tables. Conversely, the `--no-create-info` option tells **mysqldump** to suppress `CREATE` statements from the output, so that the dump file contains only table data.

- `shell> mysqldump --no-data test > dump-defs.sql`
- `shell> mysqldump --no-create-info test > dump-data.sql`

- For a definition-only dump, add the `--routines` and `--events` options to also include stored routine and event definitions.

- `shell> mysqldump --no-data --routines --events test > dump-defs.sql`

mysqldump Tips

- ▶ **Using mysqldump to Test for Upgrade Incompatibilities**
- ▶ When contemplating a MySQL upgrade, it is prudent to install the newer version separately from your current production version. Then you can dump the database and database object definitions from the production server and load them into the new server to verify that they are handled properly. (This is also useful for testing downgrades.)
- ▶ On the production server:
 - ▶ `shell> mysqldump --all-databases --no-data --routines --events > dump-defs.sql`
- ▶ On the upgraded server:
 - ▶ `shell> mysql < dump-defs.sql`

Mydumper

Mydumper is a logical backup tool that's similar to mysqlpump. Mydumper is an open source backup tool, which you need to install separately.

Mydumper has these advantages over mysqldump:

- ▶ Parallelism (hence, speed) and performance (avoids expensive character set conversion routines and has efficient code overall).
- ▶ Consistency. It maintains snapshots across all threads, provides accurate master and slave log positions, and so on. Mysqlpump does not guarantee consistency.
- ▶ Easier to manage output (separate files for tables and dumped metadata, and it is easy to view/parse data). Mysqlpump writes everything to one file, which limits the option of loading selective database objects.
- ▶ Inclusion and exclusion of database objects using regex.
- ▶ The option to kill long-running transactions that block the backup and all subsequent queries.

Full backup

- ▶ The following command takes a backup of all databases
 - ▶ shell> mysdumper -uroot --password=<password> --outputdir /backups
- ▶ Several files are created in the /backups folder.
- ▶ Each database has its CREATE DATABASE statement as <database_name>-schema-create.sql.
- ▶ Each table will have its own schema and data files.
- ▶ Schema files are stored as <database_name>.<table>-schema.sql
- ▶ Data files are stored as <database_name>.<table>.sql
- ▶ The views are stored as <database_name>.<table>-schema-view.sql.
- ▶ Stored routines, triggers, and events are stored as <database_name>-schema-post.sql

Consistent backup

- ▶ Consistent backup
 - ▶ The metadata file in the backup directory contains the binarylog coordinates for consistent backup.
 - ▶ root@master2 backups]# more metadata
- Started dump at: 2021-01-14 16:38:07
- SHOW MASTER STATUS: I
- Log: mysql-bin.000004
- Pos: 154
- GTID:
- Finished dump at: 2021-01-14 16:38:10

Backup of specific databases using regex

- ▶ You can include/exclude specific databases using the regex option.
- ▶ The following command will exclude mysql and test databases from the backup
 - ▶ `mydumper -u root --password=Not_so_secure1! --regex '^(?!(mysql|test))' --outputdir /backups/specific_dbs`
- ▶ To dump only mysql and test databases:
 - ▶ `mydumper -u root --password=Not_so_secure1! --regex '^(mysql\.|test\.)' --outputdir /backups/specific_dbs`
- ▶ To not dump all databases starting with test
 - ▶ `mydumper -u root --password=Not_so_secure1! --regex '^(?!(test))' --outputdir /backups/specific_dbs`
- ▶ <https://regexr.com/> To understand this regex combination visit mention website

Taking backup of a big table using mydumper

- ▶ To speed up the dump and restore of a big table, you can split it into small chunks. The chunk size can be specified by the number of rows it contains and each chunk will be written into a separate file.
- ▶ `mydumper -u root --password=Not_so_secure1! -B employees -T employees --triggers --events --routines --rows=10000 -t 8 --trx-consistency-only --outputdir /root/backup/employee_table_chunks`
 - ▶ `-t`: Specifies the number of threads
 - ▶ `--trx-consistency-only`: If you are using only transnational tables, such as InnoDB, using this option will minimize the locking
 - ▶ `--rows`: Split the table into chunks of this number of rows

Taking backups using mydumper

- ▶ **Compressed backups:**
- ▶ You can specify the `-compress` option to compress the backup:
 - ▶ `mydumper -u root --password=Not_so_secure1! -B employees -T employees -t 5 --trx-consistency-only --compress --outputdir /backups/employees_compress`
- ▶ **Backing up only data:**
- ▶ You can use the `--no-schemas` option to skip the schema and take a data-only backup
 - ▶ `mydumper -u root --password=Not_so_secure1! -B employees -T employees -t 4 --no-schemas --compress --trx-consistency-only --outputdir/backups/employees_data`